

SSD_OTA upgrade usage reference

1 Overview

This article introduces the upgrade solution provided by Sigmastar's platform. The process mainly includes the process of packaging, unpacking and upgrading in OTA upgrade. Support partition packaging, partition upgrade.

Upgrade package background server maintenance, terminal download and management, please combine third-party services.

2. Partition packaging process

2.1. Introduction to packaging tools

The packaging tool is executed on a Linux server, and the running environment is ubuntu 16.04. Only the required file headers will be generated based on the file currently being upgraded.

The directory where the bin of the packaging tool is located:

```
project/image/makefiletools/bin/otapack
```

otapack tool command parameter introduction:

-c --create: Create an empty upgrade package file header, this command can use -b or -e to add a script to start or end the upgrade while creating the file header.

-b --begin-script: Specify a shell script file on the server and package it into the upgrade package for the script to be executed by the board before the upgrade.

-e --end-script: Specify a shell script file on the server and package it into the upgrade package for the script to be executed by the board after the upgrade is complete.

-a --append: Append new header information to the already created upgrade package. The information about the upgrade must be informed in the new information. For the specific method, please refer to the example, or configure as needed in the remaining parameter introduction.

-s --src-file: Specifies the path to the source file on the server that needs to be updated.

-d --dst-file: Specify the target file to be updated or the path of the partition node in the target board.

-t --dst-file-size: The size of the target file or partition to be updated, in bytes, which can be expressed in 16/10 hexadecimal.

-m --file-mode: Specify the updated file permissions, refer to Linux mode_t.

--block-update: Set the source file specified in the server as raw partition data.

--ubi-update: Set the source file specified in the server as the volume of the UBI file system.

--file-update: Set the source file specified in the server to be updated as a single file.

--file-add: Set the source file specified in the server as the newly added file in the target board.

--file-delete: Only update the packaged data header, delete the specified target file on the board.

--dir-update: Scan the contents of the old and new folders, compare the differences, increase and decrease, and then pack them in batches.

--help: Print help information.

--debug: 打开调试信息。

otaunpack工具命令参数介绍：

-x: 解包并升级压缩的文件

-r: 解包并升级非压缩的文件

2.2. 一般的打包流程举例

1. 创建一个空的header

```
otapack -c SStarOta.bin
```

2. 若有需要可以用-b/-e指令添加脚本

```
otapack -c SStarOta.bin -b start.sh -e end.sh
```

3. 根据需要打包的文件更新header数据

```
otapack -a SStarOta.bin -s ./images/kernel -d /dev/mtdblock8 -t 0x500000 --block-update
```

4. 循环执行3步骤，所有的文件进行打包

5. 压缩升级包

```
gzip SStarOta.bin
```

2.3. 在ALKAID中打包介绍

OTA打包流程目前已经整合到了Makefile中。

当程序编译和打包完成后，在project下输入指令：

```
make image-ota
```

会出现如下交互界面，指定在板子上执行的脚本路径，可以选择添加或者不添加：

```
Start scripts:
```

```
End scripts:
```

在打包分区时，列出了打包的分区数据:

```
Make ipl ?(yes/no)
```

做出相关的选择后，会在 `project/image/output/images/` 下产生 `SStarOta.bin.gz`

在配置partition的config文件中会配置分区进行打包。

举例：

文件 `spinand.ubifs.p2.partition.config` 中有变量：`OTA_IMAGE_LIST`

在此变量后面追加需要打包的分区名称，并在分区的配置中添加字段 `xxx$(OTABLK)`，配置该分区需要升级的目标节点路径。

只有在 `OTA_IMAGE_LIST` 添加了分区名，才会在 `make image-ota` 的时候会询问该分区是否要进行ota升级。

分区打包脚本的所有逻辑在 `image/ota.mk` 中实现，有兴趣的可以自行研究。

2.4. 不带文件系统的RAW DATA打包

分区升级步骤大多相似，把需要升级的文件填到对应的 `mtdblock` 中，打包的流程也一样。不同的是在RAW DATA分区打包要自行实现升级包头的创建和数据填充，这里以spinand的IPL分区打包举例：

```
define makeota
@read -p "Make $(1) ?(yes/no)" select; \
if [ "${select}" == "yes" -o "${select}" == "y" ]; then \
    $(foreach
n,$(3),$(PROJ_ROOT)/image/makefiletools/bin/otabinheader -s $(2) -d
$(n) -t $(4) -p $(5) -a $(IMAGEDIR)/SStarOta.bin;cat $(2) >>
$(IMAGEDIR)/SStarOta.bin;) \
fi;
endef

ipl_spinand_mkota_ :
    $(call makeota,$(patsubst
%_spinand_mkota_,%,$@),$(IMAGEDIR)/ipl_s.bin,$($(patsubst
%_spinand_mkota_,%,$@)$(OTABLK)),,$($(patsubst
%_spinand_mkota_,%,$@)$(PATSIZE)),0)
```

2.5. ubifs/ squashfs/ jffs2打包

如果新创建了这些分区，并且把它们加到 `OTA_IMAGE_LIST` 中，则无需再ota.mk中添加特殊处理，这一类的分区升级文件会统一处理。

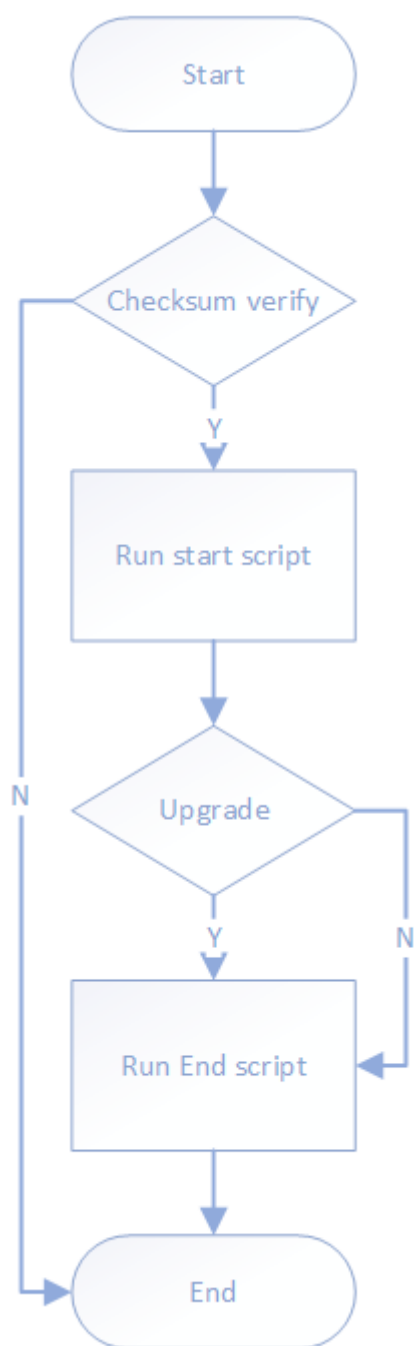
UBIFS的分区升级方式与其它两种格式稍有差异，因此在打包的时候UBIFS打包的时候请使用`--ubi-update`

2.6. 个别文件打包

个别文件打包的选项是`--file-update`，目前ALKaid中还没有专门针对文件更新的打包，因此使用者需要自己手动添加。

3. 分区更新流程

3.1. 升级流程



3.2. 分区升级前注意

在分区升级之前请确保分区umount成功。可以在start scripts脚本中作相关的umount动作。若需更新文件，请确保文件在可读写的文件系统中，并且有写权限。

otaupack工具请找FAE获取。

3.3. 压缩的升级包更新

```
otaupack -x SStar0ta.bin.gz
```

3.4. 非压缩的升级包更新

```
otaupack -r SStar0ta.bin
```

3.5. 升级UI显示

升级时可指定一张全屏的背景图片贴到framebuffer上，图片支持jpg和png格式。同时在framebuffer中绘制进度条，otaunpack在0.2版本之后支持进度条和文字UI旋转，设定-s参数0/½/3支持None/90°/180°/270°旋转。

```
otaunpack -x SStar0ta.bin.gz -p upgrade.jpg
```

3.6. 升级状态获取

在END script中利用内部变量 `OTA_STATUS` 可以获取升级的状态。

若 `OTA_STATUS` 为0则升级成功，否则为-1。

3.7. SPI-NAND坏块处理

使用--block-update命令打包升级的程序中，如果设定的是mtdblock块设备节点，这种操作不会处理坏块，推荐nor flash升级可以使用此节点，若spinand需要升级同时处理坏块，请用--block-update设定为mtd的字符设备节点，otaupack和

otaunpack应用程序在0.2版本后可以支援使用mtd-utils在升级的时候处理坏块，请在busybox中打开nand_write和flash_eraseall。